// T-OLIMPIANS — PREZENTACJA DO LEKCJI

Backtracking — szukanie z wycofywaniem

Próbuj, sprawdzaj, wycofuj się: n hetmanów krok po kroku na planszy, labirynty, sudoku, kolorowanie mapy, podział na palindromy i wykreślanka. Ślepe zaułki rozpoznajemy jak najwcześniej i obcinamy całe gałęzie.

C++ · REKURENCJA · CZĘŚĆ 5 Z 6 · PO CZĘŚCIACH 1–4



PLAN PREZENTACJI

01 Czym jest backtracking?
02 Hetmany: zasady
03 Hetmany krok po kroku
04 Hetmany: kod
05 Labirynt
06 Szczur w labiryncie
07 Sudoku
08 Kolorowanie mapy
09 Podział na palindromy
10 Wykreślanka
11 Obcinanie gałęzi
12 Sprawdź się
13 Podsumowanie

01 Backtracking — próbuj i wycofuj się

Idziesz przez labirynt z kredą. Na każdym skrzyżowaniu wybierasz jedną drogę i zaznaczasz ją kredą. Gdy trafisz na **ślepy zaułek**, wracasz do ostatniego skrzyżowania i próbujesz **innej** drogi.

Tak działa backtracking: próbuj, a gdy się nie da — wycofaj się i spróbuj inaczej.

`wybierz`
jeden z możliwych ruchów

`sprawdź`
czy ruch w ogóle jest dozwolony

`zejdź`
rekurencyjnie szukaj dalej

`cofnij`
po powrocie przywróć stan sprzed ruchu

01 Schemat backtrackingu

```
void szukaj(stan) {
    if (stan jest rozwiązaniem) {
        zapisz / wypisz / policz;
        return;
    }
    for (każdy możliwy ruch) {
        if (ruch jest niedozwolony) continue;
        wykonaj ruch;
        szukaj(nowy stan);
        cofnij ruch;
    }
}
```

To ten sam schemat **wybierz** → **zejdź** → **cofnij** co w części 4. Różnica: w części 4 prawie każda gałąź dawała wynik (każdy ciąg, każdy podzbiór). Tutaj **większość gałęzi to ślepe zaułki**.

> Klucz do szybkiego backtrackingu: sprawdzać „czy ruch jest dozwolony" **jak najwcześniej** — zanim zejdziemy głębiej.

02 Kogo bije hetman?



Hetman (królowa) w szachach bije wszystkie pola w swoim **wierszu**, w swojej **kolumnie** i na obu swoich **przekątnych** — na dowolną odległość.

> Na planszy obok hetman stoi w wierszu 2, kolumnie 2 (liczymy od 0). Pomarańczowe pola są przez niego bite.

02 Problem n hetmanów

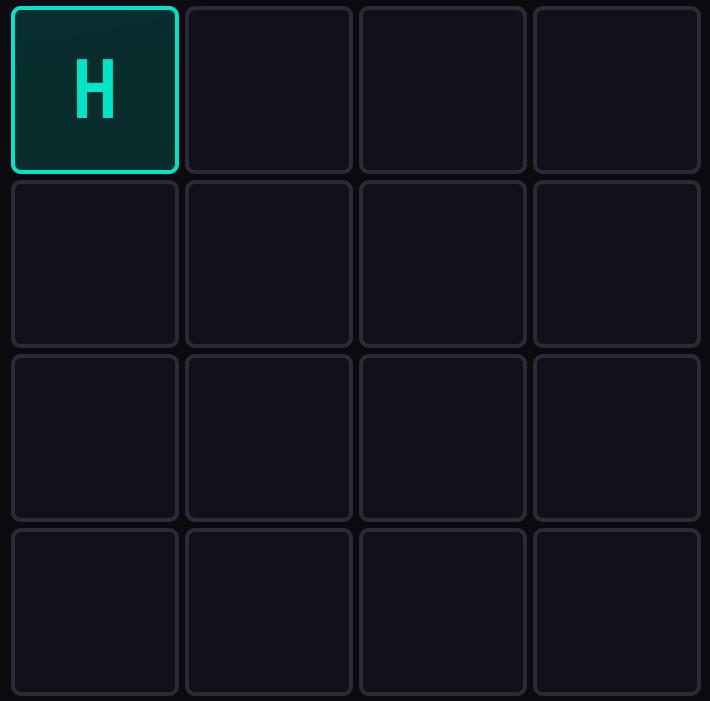
Ustaw **n** hetmanów na szachownicy **n × n** tak, żeby żadne dwa się nie biły. Ile jest takich ustawień? Jak wygląda pierwsze z nich?

1. W każdym wierszu musi stać **dokładnie jeden** hetman (dwa w jednym wierszu by się biły).
2. Więc decyzja dla wiersza w to: **w której kolumnie** postawić hetmana?
3. Zapamiętamy to w tablicy **kol[w]**.



Jedno z dwóch ustawień dla n = 4: **kol = {1, 3, 0, 2}**.

03 Wiersz 0: hetman w kolumnie 0



Zaczynamy od pierwszej możliwości: wiersz 0, kolumna 0. Nic jeszcze nie stoi na planszy, więc to pole jest bezpieczne.

`kol`	{0, ·, ·, ·}

03 Wiersz 1: kolumny 0 i 1 są bite, 2 jest wolna



W wierszu 1 próbujemy kolumn od lewej: kolumnę 0 bije hetman z góry (ta sama kolumna), kolumnę 1 — po przekątnej. Kolumna 2 jest wolna — **stawiamy**.

`kol`	{0, 2, ·, ·}

03 Wiersz 2: wszystko bite — ślepy zaułek!



W wierszu 2 **każda** kolumna jest bita: 0 i 2 — przez kolumny, 1 i 3 — przez przekątne hetmana z wiersza 1. Nie ma gdzie postawić hetmana. **Wycofujemy się** do wiersza 1.

`kol`	{0, 2, ·, ·}

03 Powrót do wiersza 1: próbujemy kolumny 3



Cofamy hetmana z kolumny 2 (kol[1] znów pusta) i próbujemy następnej kolumny: 3. Jest wolna — stawiamy i schodzimy do wiersza 2.

kol	{0, 3, ·, ·}

03 Wiersz 2: kolumna 1



W wierszu 2 kolumna 0 jest bita, ale kolumna 1 — wolna. Stawiamy i schodzimy do wiersza 3.

`kol`	{0, 3, 1, ·}

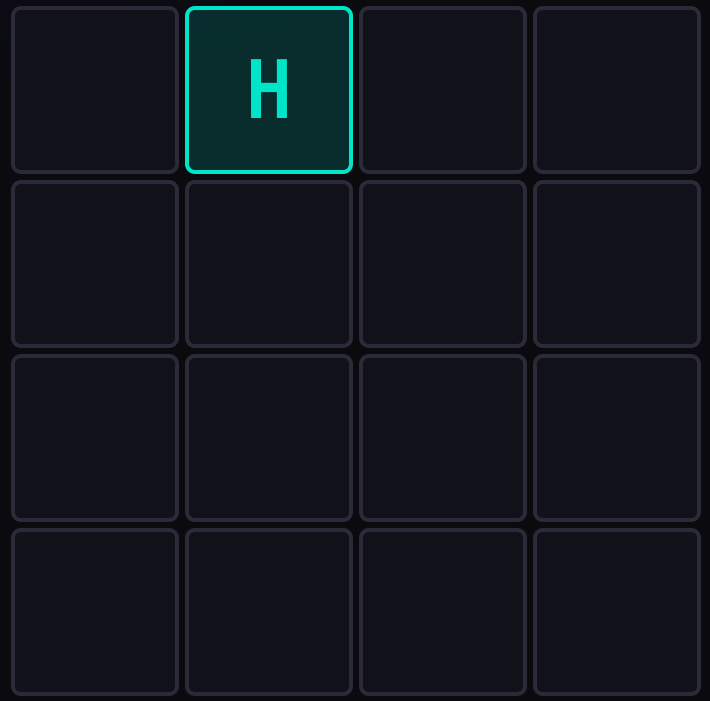
03 Wiersz 3: znowu ślepy zaułek



W wierszu 3 wszystkie kolumny są bite. Wycofujemy się do wiersza 2 — ale tam kolumny 2 i 3 też są bite. Wracamy do wiersza 1 — nie ma już kolumn do sprawdzenia. Wracamy aż do **wiersza 0**.

`kol`	{0, 3, 1, ·}

03 Wiersz 0: kolumna 1



Hetman w rogu (kolumna 0) nie prowadzi do żadnego rozwiązania — sprawdziliśmy całe jego poddrzewo. Przesuwamy go na kolumnę 1 i zaczynamy od nowa niżej.

kol	{1, ·, ·, ·}

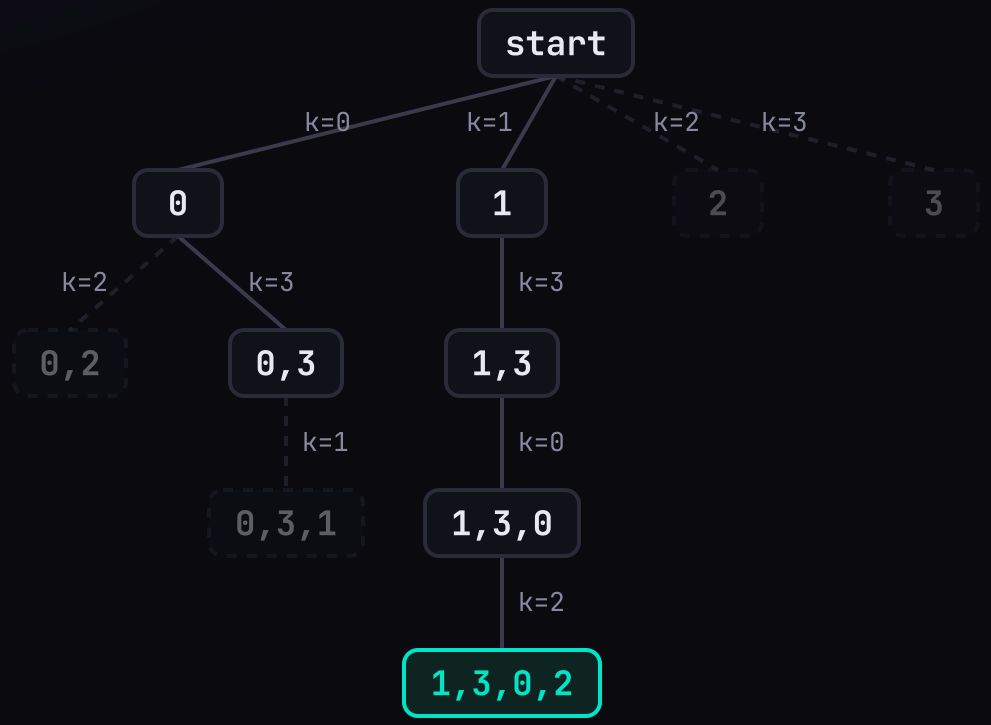
03 Wiersze 1, 2, 3: wszystko pasuje!



Wiersz 1 → kolumna 3, wiersz 2 → kolumna 0, wiersz 3 → kolumna 2. Wszystkie cztery hetmany stoją i żaden nikogo nie bije — **rozwiązanie**!

`kol`	{1, 3, 0, 2}

03 Całe szukanie jako drzewo



Węzeł to ustawienie kilku pierwszych wierszy (kol). Wyblakłe węzły z przerywaną ramką to **ślepe zaułki** — tam w kolejnym wierszu nie było wolnej kolumny.

> Gałęzie zaczynające się od 2 i 3 dają drugie rozwiązanie {2, 0, 3, 1} — lustrzane odbicie pierwszego.

04 Czy pole jest bezpieczne?

```
int n, kol[20];

bool bezpieczny(int w, int k) {
    for (int i = 0; i < w; i++) {
        if (kol[i] == k) return false;
        if (abs(kol[i] - k) == w - i) return false;
    }
    return true;
}
```

Hetman w polu (w, k) jest bezpieczny, jeśli żaden hetman z wcześniejszych wierszy i = 0, …, w − 1 go nie bije:

1. ta sama kolumna: `kol[i] == k`,
2. ta sama przekątna: odległość w poziomie równa odległości w pionie — `abs(kol[i] - k) == w - i`.

Wierszy sprawdzać nie trzeba — z budowy rozwiązania każdy hetman stoi w innym wierszu.

04 Hetmany — liczymy wszystkie ustawienia

```
int licznik = 0;

void ustaw(int w) {
    if (w == n) {
        licznik++;
        return;
    }
    for (int k = 0; k < n; k++) {
        if (!bezpieczny(w, k)) continue;
        kol[w] = k;
        ustaw(w + 1);
    }
}
```

Gdy `w == n`, hetmany stoją we wszystkich wierszach — mamy rozwiązanie.

Cofanie jest tu „ukryte”: `kol[w]` zostanie po prostu **nadpisane** przy następnej kolumnie, a wiersze poniżej w i tak sprawdzamy tylko do w − 1. Dlatego nie trzeba nic czyścić.

n	4	5	6	7	8	9	10
ustawień	2	10	4	40	92	352	724

04 Pierwsze ustawienie — i koniec szukania

```
bool ustaw(int w) {
    if (w == n) return true;
    for (int k = 0; k < n; k++) {
        if (!bezpieczny(w, k)) continue;
        kol[w] = k;
        if (ustaw(w + 1)) return true;
    }
    return false;
}
```

Wzorzec „znajdź jedno” z części 4: gdy głębsze wywołanie zwróci **true**, przekazujemy true w górę i **nie próbujemy** już kolejnych kolumn.



Pierwsze znalezione ustawienie dla n = 6.

04 Ile pracy oszczędza backtracking?

sposób	ile ustawień sprawdzamy (n = 8)
dowolne 8 pól z 64	ok. 4,4 miliarda
po jednym hetmanie w wierszu, bez sprawdzania	$8^8 \approx 16{,}7$ miliona
backtracking (sprawdzamy od razu)	**ok. 2 tysiące węzłów**

Każdy ślepy zaułek odcina **całe poddrzewo**. Gdy hetman w wierszu 2 jest bity, nie ustawiamy już wierszy 3, 4, …, 7 — ani razu.

05 Ile dróg przez labirynt?



Idziemy z lewego górnego rogu do prawego dolnego: w górę, w dół, w lewo, w prawo, **bez ścian** i **bez odwiedzania pola dwa razy**. Ile jest dróg?

> **Uwaga:** Tym razem wolno chodzić we wszystkie strony, więc memoizacja z części 3 nie zadziała: liczba dróg z pola zależy od tego, **które pola są już zajęte**.

05 Labirynt — kod

```
bool odw[6][6];
long long licznik = 0;

void idz(int i, int j) {
    if (i < 0 || j < 0 || i >= n || j >= m) return;
    if (g[i][j] == '#' || odw[i][j]) return;
    if (i == n - 1 && j == m - 1) {
        licznik++;
        return;
    }
    odw[i][j] = true;
    idz(i + 1, j); idz(i - 1, j);
    idz(i, j + 1); idz(i, j - 1);
    odw[i][j] = false;
}
```

1. poza planszą, ściana albo pole już na drodze — **ślepy zaułek**,
2. meta — **jeszcze jedna droga**,
3. **zaznacz** pole, idź w cztery strony, **odznacz** pole.

Odznaczenie to cofnięcie wyboru: inne drogi mogą to pole odwiedzić — po swojemu.

05 Droga w trakcie szukania



Fioletowe pola są **zaznaczone** (odw = true) — leżą na obecnej drodze i na stosie. Z pola (1, 3) w dół jest ściana (2, 3), w lewo — ściana (1, 2). W górę — pole już odwiedzone.

Ślepy zaułek: funkcja dla (1, 3) kończy się, **odznacza** pole i wraca do (0, 3), które próbuje kolejnych kierunków.

06 Szczur w labiryncie — wypisz drogi



WSZYSTKIE DROGI (ALFABETYCZNIE)

```
DDRDRR
DRDDRR
```

Zadanie Strivera (L19): 1 — wolne, 0 — ściana. Drogi zapisujemy literami **D** (dół), **L** (lewo), **R** (prawo), **U** (góra) i wypisujemy **alfabetycznie**.

```
const int di[4] = {1, 0, 0, -1};
const int dj[4] = {0, -1, 1, 0};
const string lit = "DLRU";
```

Jeśli w każdym polu próbujemy kierunków w kolejności **D, L, R, U**, drogi wychodzą od razu posortowane — bez sortowania na końcu.

06 Szczur — kod

```
void szukaj(int i, int j, string droga) {
    if (i == n - 1 && j == n - 1) { cout << droga << endl; return; }
    odw[i][j] = true;
    for (int d = 0; d < 4; d++) {
        int a = i + di[d], b = j + dj[d];
        if (a < 0 || b < 0 || a >= n || b >= n) continue;
        if (g[a][b] == 0 || odw[a][b]) continue;
        szukaj(a, b, droga + lit[d]);
    }
    odw[i][j] = false;
}
```

Droga jest **parametrem** (`droga + lit[d]` tworzy nowy napis), więc jej nie trzeba cofać. Tablicę **odw** — wspólną — trzeba.

07 Sudoku

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

Wypełnij puste pola cyframi 1–9 tak, żeby w każdym **wierszu**, każdej **kolumnie** i każdym **kwadracie 3 × 3** każda cyfra była dokładnie raz.

1. idź po polach od lewej do prawej, wiersz po wierszu,
2. na puste pole próbuj cyfr 1, 2, …, 9 — tylko takich, które **pasują**,
3. dla każdej pasującej schodź do następnego pola,
4. gdy nic nie pasuje — **wycofaj się** i zmień cyfrę wcześniej.

07 Czy cyfra pasuje?

```
int g[9][9];

bool pasuje(int w, int k, int d) {
    for (int i = 0; i < 9; i++) {
        if (g[w][i] == d) return false;   // wiersz
        if (g[i][k] == d) return false;   // kolumna
    }
    int bw = w - w % 3, bk = k - k % 3;   // róg kwadratu
    for (int a = 0; a < 3; a++)
        for (int b = 0; b < 3; b++)
            if (g[bw + a][bk + b] == d) return false;
    return true;
}
```

Lewy górny róg kwadratu 3 × 3, w którym leży pole (w, k), to (`w - w % 3` , `k - k % 3`).

pole	róg kwadratu
(4, 7)	(3, 6)
(0, 2)	(0, 0)
(8, 5)	(6, 3)

07 Sudoku — rozwiązywanie

```
bool rozwiaz(int p) {          // p = 0 … 80
    if (p == 81) return true;
    int w = p / 9, k = p % 9;
    if (g[w][k] != 0) return rozwiaz(p + 1);
    for (int d = 1; d <= 9; d++) {
        if (!pasuje(w, k, d)) continue;
        g[w][k] = d;
        if (rozwiaz(p + 1)) return true;
        g[w][k] = 0;
    }
    return false;
}
```

Pola numerujemy od 0 do 80: wiersz to p / 9, kolumna p % 9. Pole już wypełnione przeskakujemy.

Gdy rozwiązanie się znajdzie, `true` wraca w górę przez wszystkie wywołania i plansza zostaje **wypełniona**. Gdy cyfra prowadzi w ślepy zaułek — `g[w][k] = 0` czyści pole przed następną próbą.

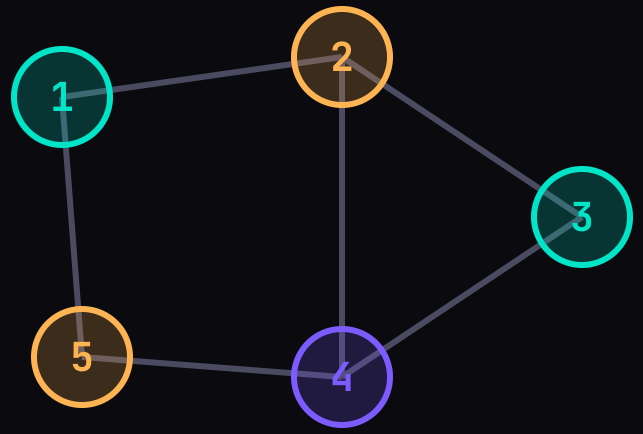
07 Rozwiązane sudoku

5	3	4	6	7	8	9	1	2
6	7	2	1	9	5	3	4	8
1	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9	2	4	8	5	6
9	6	1	5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9

Turkusowe cyfry wstawił backtracking. Dla typowego sudoku z gazety wystarczy kilka tysięcy wywołań — bo prawie każda zła cyfra „wysypuje się” po kilku polach.

> Ten sam schemat rozwiązuje wiele łamigłówek: krzyżówki liczbowe, kakuro, ułożenie klocków…

08 Kolorowanie mapy



Państwa to kółka, kreska oznacza, że dwa państwa **sąsiadują**. Pomaluj każde państwo jednym z k kolorów tak, żeby sąsiedzi mieli **różne** kolory. Na ile sposobów?

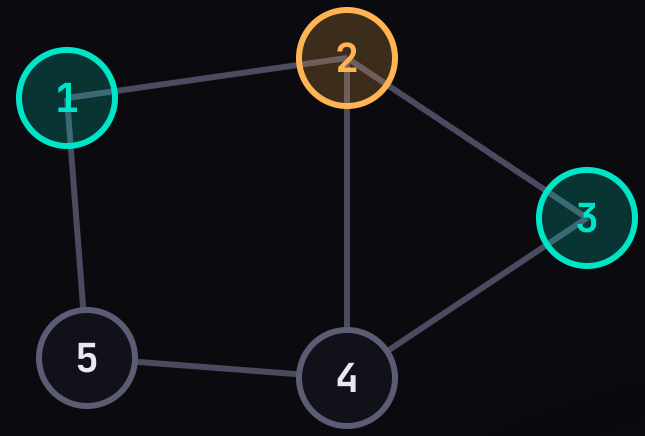
Koloruj państwa po kolei: państwu v nadaj kolor, którego nie ma **żaden już pokolorowany sąsiad**, i przejdź do v + 1.

> U Strivera: L16 — *M-Coloring Problem*.

08 Kolorowanie — kod

```
vector<int> sas[11];
int kolor[11];          // 0 = jeszcze bez koloru
long long licznik = 0;

void koloruj(int v) {
    if (v > n) { licznik++; return; }
    for (int c = 1; c <= k; c++) {
        bool wolny = true;
        for (int u : sas[v])
            if (kolor[u] == c) wolny = false;
        if (!wolny) continue;
        kolor[v] = c;
        koloruj(v + 1);
        kolor[v] = 0;
    }
}
```

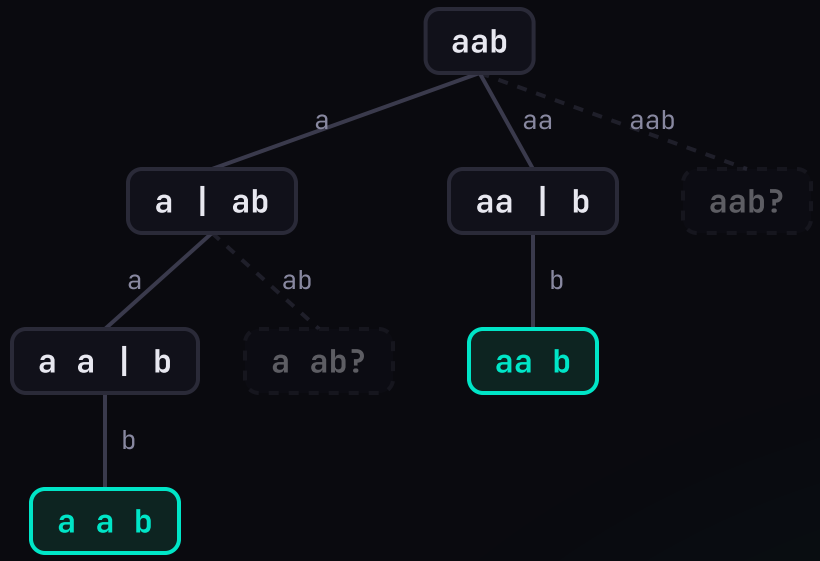


W trakcie: państwa 1, 2, 3 pokolorowane. Państwo 4 sąsiaduje z 2 i 3, więc może dostać tylko kolor, którego nie mają one.

09 Podział na palindromy

Potnij słowo na kawałki tak, żeby **każdy kawałek był palindromem**. Wypisz wszystkie sposoby. Dla „aab”: **a a b** oraz **aa b**.

Funkcja tnij(i) wybiera **pierwszy kawałek** zaczynający się na pozycji i: najpierw najkrótszy, potem coraz dłuższe. Kawałek, który nie jest palindromem, to ślepy zaułek.



09 Podział na palindromy — kod

```
void tnij(int i) {
    if (i == n) { wypisz(kawalki); return; }
    for (int j = i; j < n; j++) {
        if (!palindrom(i, j)) continue;
        kawalki.push_back(s.substr(i, j - i + 1));
        tnij(j + 1);
        kawalki.pop_back();
    }
}
```

`palindrom(i, j)` sprawdza kawałek od i do j — możesz użyć rekurencyjnej funkcji z części 2.

SŁOWO „KAJAK”

```
k a j a k
k aja k
kajak
```

U Strivera: L17 — *Palindrome Partitioning*.

10 Wykreślanka



Słowo jest ukryte, jeśli da się je przeczytać, idąc po **sąsiednich polach** (góra, dół, lewo, prawo), każde pole najwyżej raz. „abcced” jest ukryte (fioletowa droga).

```
bool szukaj(int i, int j, int k) {
    if (poza planszą || uzyte[i][j]) return false;
    if (g[i][j] != w[k]) return false;
    if (k == w.size() - 1) return true;
    uzyte[i][j] = true;
    bool ok = szukaj(i + 1, j, k + 1) || szukaj(i - 1, j, k + 1)
           || szukaj(i, j + 1, k + 1) || szukaj(i, j - 1, k + 1);
    uzyte[i][j] = false;
    return ok;
}
```

Operator `||` sam przerywa sprawdzanie przy pierwszym **true** — kolejne kierunki nie są już próbowane.

11 Obcinanie gałęzi — im wcześniej, tym lepiej

Gdy wiesz, że z danego miejsca nie da się dojść do rozwiązania — nie schodź głębiej.

Zadanie	Obcinamy, gdy…
hetmany	nowy hetman jest bity — nie ustawiamy dalszych wierszy
plecak	waga przekroczyła pojemność — dokładanie nic nie pomoże
liczby o sumie cyfr s	suma już większa od s albo nawet same dziewiątki nie dobiją do s
suma podzbioru (liczby dodatnie)	uzbierana suma przekroczyła cel

11 Przykład: liczby o danej sumie cyfr

```
void buduj(int i, int suma, string liczba) {
    if (suma > s) return;                  // za dużo
    if (suma + 9 * (n - i) < s) return;    // za mało
    if (i == n) { cout << liczba << endl; return; }
    for (int d = (i == 0 ? 1 : 0); d <= 9; d++)
        buduj(i + 1, suma + d, liczba + char('0' + d));
}
```

Wszystkie n-cyfrowe liczby o sumie cyfr s. Dwa warunki obcinają gałęzie, które na pewno nie dadzą wyniku:

1. suma już **przekroczyła** s,
2. nawet same **dziewiątki** na pozostałych n − i pozycjach nie wystarczą.

N = 3, S = 2

```
101
110
200
```

11 Przykład: plecak — sprawdź wszystko, ale mądrze

```
void pakuj(int i, long long waga, long long wartosc) {
    if (waga > W) return;           // obcinanie
    if (i == n) {
        najlepsza = max(najlepsza, wartosc);
        return;
    }
    pakuj(i + 1, waga, wartosc);
    pakuj(i + 1, waga + w[i], wartosc + v[i]);
}
```

Bierz / nie bierz z części 4 + zapamiętywanie **najlepszego** wyniku. Dla n ≤ 20 to najwyżej ok. 2 miliony wywołań.

> Gdy wag jest dużo i są małe, plecak rozwiązuje się **programowaniem dynamicznym** — w kompendium temat „Plecak 0-1". Backtracking wygrywa, gdy przedmiotów jest mało, a wagi są ogromne.

12 Pytanie 1: ile ustawień ma 4 hetmanów?



NA EKRANIE

?

Zastanów się i zapisz odpowiedź, **zanim** przejdziesz dalej.

12 Odpowiedź 1: dwa ustawienia





{1, 3, 0, 2} i {2, 0, 3, 1} — jedno jest lustrzanym odbiciem drugiego.

12 Pytanie 2: co się stanie bez odznaczania?

```
void idz(int i, int j) {
    if (poza || g[i][j] == '#' || odw[i][j]) return;
    if (meta) { licznik++; return; }
    odw[i][j] = true;
    idz(i + 1, j); idz(i - 1, j);
    idz(i, j + 1); idz(i, j - 1);
    // brak: odw[i][j] = false;
}
```

NA EKRANIE

?

Zastanów się i zapisz odpowiedź, **zanim** przejdziesz dalej.

12 Odpowiedź 2: znajdzie za mało dróg

Uwaga: Pole odwiedzone przez **jedną** drogę zostaje zablokowane dla **wszystkich następnych**. Program policzy tylko część dróg — zwykle jedną albo dwie — choć nie zgłosi żadnego błędu.

Każde zaznaczenie wspólnego stanu musi mieć swoje **odznaczenie** po powrocie — tak samo jak push_back i pop_back w części 4.

12 Pytanie 3: czy hetman (3, 1) jest bezpieczny?



Hetmany stoją w (0, 0) i (1, 3). Sprawdzamy pole (3, 1).

NA EKRANIE

?

Zastanów się i zapisz odpowiedź, **zanim** przejdziesz dalej.

12 Odpowiedź 3: nie — przekątna z (1, 3)

hetman	ta sama kolumna?	|Δ kolumn| == Δ wierszy?
(0, 0)	0 ≠ 1	|0 − 1| = 1, 3 − 0 = 3 → nie
(1, 3)	3 ≠ 1	|3 − 1| = **2**, 3 − 1 = **2** → **tak!**



13 Ściąga: backtracking

Element	W kodzie	Pamiętaj
stan	tablice kol[], odw[][], g[][], kolor[], wektor wybranych	wspólny dla wszystkich wywołań
ruch dozwolony?	`if (!bezpieczny(...)) continue;`	sprawdzaj przed zejściem
wybierz → zejdź → cofnij	`odw = true; idz(...); odw = false;`	każda zmiana stanu → odwróć
znajdź jedno	`if (szukaj(...)) return true;`	true kończy całe szukanie
policz / wypisz wszystkie	licznik++ albo cout w liściu	przejdzie całe (obcięte) drzewo
obcinanie	return, gdy rozwiązania już się nie da dokończyć	im wcześniej, tym szybciej

13 Zadania na koniec — Twoja kolej

Zadania są na **t-olimpians.com** → **Kompendium** → **Funkcje rekurencyjne** → **Krok 5**. Każde ma sprawdzarkę i wskazówkę:

1. Hetmany — ile ustawień?
2. Hetmany — pierwsze ustawienie
3. Ile dróg przez labirynt?
4. Szczur w labiryncie
5. Sudoku
6. Kolorowanie mapy
7. Podział na palindromy
8. Liczby o danej sumie cyfr
9. Wykreślanka
10. Plecak — sprawdź wszystko
11. Sprawiedliwy podział

// KONIEC CZĘŚCI 5

Ślepe zaułki już Cię nie zatrzymają!

Wybierz, sprawdź, zejdź, cofnij — i obcinaj gałęzie, które nie mogą dać rozwiązania.
Zrób zadania z kroku 5, a potem część 6: dziel i zwyciężaj.

